Week 14
Offline RL: Distribution Shift, Conservatism, and Implicit Q-Learning

STUDY-RL — a self-paced Deep RL curriculum

October 20, 2026

Contents

Learning objectives 3

1 Motivation & intuition 4

2 Mathematical formulation 5
- 2.1 Extrapolation error . . . 5
- 2.2 BCQ and BEAR — behaviour regularisation . . . 5
- 2.3 CQL — conservative Q-learning . . . 6
- 2.4 IQL — implicit Q-learning . . . 6
- 2.5 AWR / AWAC policy extraction . . . 7
- 2.6 TD3+BC and ReBRAC . . . 7
- 2.7 Off-policy evaluation (OPE) primer . . . 7

3 Algorithms 9
- 3.1 Worked numerical example . . . 9

4 Implementation walkthrough 10

5 Empirical results 11

6 Exercises 13

7 Further reading 14

Learning objectives

Study companion. Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

Key Idea

After this week you will be able to:

1. Define the offline RL setting and explain why standard Q-learning fails on logged data through *extrapolation error*.
2. Derive the BCQ behaviour-support constraint and the BEAR MMD constraint from the Bellman backup.
3. Reproduce the CQL conservative penalty from a min-max objective and explain its policy-value lower-bound result and its estimation and regularisation assumptions.
4. Derive the IQL expectile value loss and the AWR/AWAC closed-form policy update.
5. Implement IQL (tabular) and CQL on a synthetic logged GridWorld and show that IQL beats naive Q-learning.
6. Estimate V^{π} from a logged dataset via SNIPS and discuss the OPE-FQE-DICE tradeoffs.
7. Translate offline RL pitfalls into concrete failure modes for recruitment matching deployments.

1 Motivation & intuition

In every previous week the agent generated its own data: the policy that produced the sample is the policy we are trying to improve. Offline RL inverts this. We are given a static dataset $\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^{N}$ produced by an unknown behaviour policy π_β and must return a policy π that performs well in the real environment *without any new interaction*.

Recall: PPO, SAC, DQN all trained from a buffer of *their own* transitions.

This is the practical setting of essentially every commercial RL deployment we have encountered: clickstreams from a recommender, broker–lead pairings logged in a CRM, chat trajectories from a deployed assistant. The hard part is not learning per se — it is that the Bellman backup quietly queries actions that have *never been taken in* $\mathcal{D}$. Function approximators happily extrapolate, but in the wrong direction.

Intuition. Imagine fitting a regression on $x \in [0, 1]$ and then asking it to predict at $x = 10$. You will get a number, but it is a fiction. Off-policy Q-learning maximises over actions: $\max_a Q(s', a)$ pulls towards whichever a happens to have the largest *extrapolated* value. The Bellman backup then propagates that fiction backward through the trajectory, making everything worse on the next sweep. Offline RL is the discipline of **not asking**.

Common pitfall

"Offline RL" is sometimes confused with "batch RL". They are the same: a fixed log, no fresh rollouts. The *name* "offline" is to emphasise the deployment constraint — you cannot try a new candidate broker just to learn from the click.

2 Mathematical formulation

Definition 2.0.1 (Offline RL problem)**.** Given $\mathcal{D} = \{(s_i, a_i, r_i, s_i')\}_{i=1}^N$ drawn from d^{π_β}, discount $\gamma \in (0,1)$, and an MDP $(\mathcal{S}, \mathcal{A}, P, R)$ that the learner *cannot query*, find a policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ that maximises $J(\pi) = \mathbb{E}_{s_0 \sim \rho_0}[V^\pi(s_0)]$.

Definition 2.0.2 (Distribution shift)**.** Let $d^\pi(s,a) = (1-\gamma)\sum_{t \geq 0} \gamma^t \mathbb{P}(s_t = s, a_t = a \mid \pi)$ be the discounted occupancy. *Distribution shift* is the divergence between d^{π_β} (coverage in the data) and d^π (states the learner would visit), measured by D_{TV}, D_{KL}, or χ^2.

2.1 Extrapolation error

Define the true optimality Bellman backup

$$\mathcal{B}^* Q(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P}\big[\max_{a' \in \mathcal{A}} Q(s', a')\big].$$

The action a' realising the maximum may be *out-of-distribution* with respect to $\pi_\beta(\cdot \mid s')$. Under a function approximator with capacity θ, the gradient $\nabla_\theta Q(s', a')$ "moves" the learned Q everywhere that the network thinks a' looks like another action. We get a cascading optimistic bias.

Theorem 2.1.1 (Bellman backup error decomposition — Fujimoto et al. 2019)**.** *For any Q and any state–action (s,a), write the true policy-evaluation operator as $\mathcal{B}^\pi$ and its empirical estimate as $\hat{\mathcal{B}}^\pi$. Then*

$$\begin{aligned} |Q(s,a) - Q^\pi(s,a)| \leq\ & |Q(s,a) - \hat{\mathcal{B}}^\pi Q(s,a)| \\ & + |(\hat{\mathcal{B}}^\pi - \mathcal{B}^\pi) Q(s,a)| \\ & + \gamma\, \mathbb{E}_{s' \sim P, a' \sim \pi}[|Q(s', a') - Q^\pi(s', a')|]. \end{aligned}$$

Sketch. Insert and subtract both the empirical backup $\hat{\mathcal{B}}^\pi Q$ and true backup $\mathcal{B}^\pi Q$ and use the γ-contraction of $\mathcal{B}^\pi$. □

The first term is an empirical residual; the second is model or sampling error. Neither is controlled on unsupported state–action pairs. The propagated-error term *integrates over* π, including on out-of-support actions. The whole offline literature is methods that keep $a' \sim \pi$ close enough to π_β that propagated error remains controllable.

2.2 BCQ and BEAR — behaviour regularisation

BCQ (Fujimoto 2019) trains a conditional VAE $G_\omega(a \mid s)$ on $\mathcal{D}$ and only considers actions sampled from G_ω plus a small perturbation network $\xi_\phi(s,a)$:

$$\pi_{\mathrm{BCQ}}(s) = \operatorname*{arg\,max}_{a_i + \xi_\phi(s, a_i) : a_i \sim G_\omega(\cdot \mid s)} Q_\theta(s, a_i).$$

BEAR replaces the constraint by a sampled MMD divergence $\mathrm{MMD}^2(G_\omega(\cdot|s), \pi(\cdot|s)) \leq \varepsilon$, enforced via a Lagrangian penalty on the actor.

2.3 CQL — conservative Q-learning

Definition 2.3.1 (CQL objective, discrete actions)**.** For a parameter $\alpha > 0$,

$$\mathcal{L}_{\mathrm{CQL}}(\theta) = \alpha\, \mathbb{E}_{s\sim\mathcal{D}}[\log \textstyle\sum_a \exp Q_\theta(s,a) - \mathbb{E}_{a\sim\pi_\beta(\cdot|s)} Q_\theta(s,a)] \\ + \tfrac{1}{2}\, \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}[(Q_\theta(s,a) - \mathcal{B}^\pi \hat{Q}(s,a))^2].$$

Remark (Conservatism: intuition versus guarantee). Kumar et al. (2020) establish policy-value lower bounds for specified CQL policy-evaluation variants under their tabular, coverage, sampling-error, and regularisation conditions. This is not a pointwise guarantee for every learned $Q(s,a)$, every penalty coefficient, or an arbitrary neural optimiser.

Penalty-gradient derivation (not a lower-bound proof). At a fixed state let b_a be the empirical behaviour frequency, y_a a frozen Bellman target, and $\mu_a = \exp Q_a / \sum_j \exp Q_j$. The statewise objective is

$$\ell(Q) = \alpha[\log \sum_a e^{Q_a} - \sum_a b_a Q_a] + \tfrac{1}{2} \sum_a b_a (Q_a - y_a)^2.$$

Differentiation gives

$$\frac{\partial \ell}{\partial Q_a} = \alpha(\mu_a - b_a) + b_a(Q_a - y_a).$$

For $b_a > 0$, stationarity implies $Q_a = y_a - \alpha(\mu_a/b_a - 1)$. The factor b_a in the squared-error gradient is essential. The penalty pushes down actions overweighted by μ relative to the log; it cannot, by itself, prove a policy-value bound for every target policy.

Common pitfall

Setting α too large makes the policy degenerate to imitation; too small lets extrapolation error in. The standard recipe is to *tune α with a Lagrangian* on the budget $\mathcal{B}_{\mathrm{CQL}} = \log Z - \mathbb{E}_{\pi_\beta} Q \le \tau$.

2.4 IQL — implicit Q-learning

IQL never explicitly samples actions from π; instead it estimates the upper expectile of $Q(s,a)$ for $a \sim \pi_\beta$ and uses that as a value baseline.

Definition 2.4.1 (Expectile loss)**.** For $\tau \in (0,1)$ and residual $u = y - \hat{y}$,

$$L_2^\tau(u) = |\tau - \mathbf{1}\{u < 0\}| \cdot u^2.$$

The expectile is the minimiser of $\mathbb{E}[L_2^\tau(Y - v)]$ over v.

Theorem 2.4.2 (IQL value backup)**.** *In the finite-action tabular setting with exact optimisation and bounded, fixed $Q_{\hat{\theta}}$, let V_ψ solve $\min_\psi \mathbb{E}_{(s,a)\sim\mathcal{D}}[L_2^\tau(Q_{\hat{\theta}}(s,a) - V_\psi(s))]$. Then for $\tau \to 1$, $V_\psi(s) \to \max_{a\in\operatorname{supp}\pi_\beta(\cdot|s)} Q_{\hat{\theta}}(s,a)$.*

Sketch. Fix s; we are estimating the τ-expectile of the conditional distribution of $Q(s,a)$ under $a \sim \pi_\beta(\cdot|s)$. The τ-expectile of a bounded random variable converges to its essential supremum as $\tau \to 1$. So the value becomes the in-support max — never the global max, which is exactly what we wanted to avoid. □

$$\boxed{L_V(\psi) = \mathbb{E}_{\mathcal{D}}[L_2^\tau(Q_{\hat{\theta}}(s,a) - V_\psi(s))], \quad L_Q(\theta) = \mathbb{E}_{\mathcal{D}}[(r + \gamma V_\psi(s') - Q_\theta(s,a))^2]}$$

2.5 AWR / AWAC policy extraction

Given $A(s,a) = Q(s,a) - V(s)$, AWAC solves

$$\pi^\star = \arg\max_\pi \mathbb{E}_{(s,a)\sim\mathcal{D}}[\log \pi(a|s) \cdot \exp(\beta A(s,a))],$$

which is a weighted maximum-likelihood projection of the non-parametric improvement below. The closed-form distribution, rather than a neural network fit, solves the corresponding KL-penalised problem; a hard constraint additionally requires choosing its multiplier:

$$\max_\pi \mathbb{E}_{s\sim\mathcal{D},a\sim\pi}[A(s,a)]$$
$$\text{s.t. } \mathrm{KL}(\pi(\cdot|s) \,\|\, \pi_\beta(\cdot|s)) \le \varepsilon.$$

The Lagrangian is $A(s,a) - (1/\beta)\log(\pi/\pi_\beta) + \lambda$. Setting the derivative w.r.t. $\pi(a|s)$ to zero gives $\pi^\star(a|s) \propto \pi_\beta(a|s)\exp(\beta A(s,a))$; because π_β is the empirical distribution of actions in $\mathcal{D}$, the maximum-likelihood policy regression on weighted samples is the same fit.

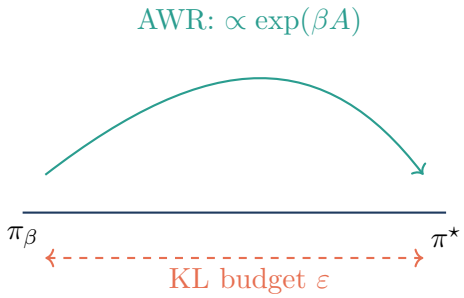


Figure 2.1: AWR/AWAC: stay inside a KL ball of π_β, reweight by exponentiated advantage.

2.6 TD3+BC and ReBRAC

TD3+BC minimises the actor loss

$$L_\pi = \mathbb{E}_{(s,a)\sim\mathcal{D}}[-\lambda Q(s,\pi(s)) + \|\pi(s) - a\|^2], \qquad \lambda = \alpha/\mathbb{E}_\mathcal{D}|Q(s,\pi(s))|.$$

The inverse-Q scale multiplies the Q *term*, with its denominator detached from differentiation. Multiplying the whole loss by $1/\lambda$ instead puts a proportional-to-$|Q|$ coefficient on imitation. These equivalent forms must not be mixed. See Fujimoto and Gu (2021). ReBRAC adds implementation choices including deeper networks, critic LayerNorm, larger batches and separately controlled actor/critic behaviour penalties. Its reported benchmark comparison is not a timeless ranking of algorithms.

2.7 Off-policy evaluation (OPE) primer

Let π be a target policy and π_β the behaviour policy. The **importance-sampling** estimator of V^π is

$$\hat{V}^\pi_{\mathrm{IS}} = \frac{1}{n}\sum_{i=1}^{n} \rho_i\, G_i, \qquad \rho_i = \prod_{t=0}^{T_i-1} \frac{\pi(a_t \mid s_t)}{\pi_\beta(a_t \mid s_t)}.$$

Trajectory-product weights can have variance that grows rapidly with horizon when target and logger differ; this is a risk, not an equality for every MDP. **Self-normalised IS (SNIPS)** uses $\sum_i \rho_i G_i / \sum_i \rho_i$ when the denominator is positive. It has finite-sample bias and can reduce variance, without universally dominating IS. **FQE** is direct, model-free policy-value regression by repeated

Bellman backups, not learning a transition model. **DICE** methods estimate occupancy ratios and avoid explicit trajectory products, but still require coverage and approximation assumptions; their estimation error does not become independent of discount or effective horizon.

Common pitfall

A deterministic target is valid for IS when the logger supports its actions. It may leave very few nonzero trajectory weights. Smoothing the target changes the policy being evaluated and can improve overlap in observed trajectories; it cannot create logger support. There is no universal 50-step cutoff.

3 Algorithms

Algorithm 1: IQL — Implicit Q-Learning

Input: Dataset $\mathcal{D}$, expectile τ, AWR temperature β, learning rates η_v, η_q, η_π.
Initialise $V_\psi, Q_\theta, Q_{\hat{\theta}} \leftarrow Q_\theta, \pi_\phi$;
for $t = 1, \ldots, T$ **do**
  Sample minibatch $\{(s_i, a_i, r_i, s'_i)\}_{i=1}^{B} \sim \mathcal{D}$;
  $\psi \leftarrow \psi - \eta_v \nabla_\psi \frac{1}{B} \sum_i L_2^\tau(Q_{\hat{\theta}}(s_i, a_i) - V_\psi(s_i))$;
  $\theta \leftarrow \theta - \eta_q \nabla_\theta \frac{1}{B} \sum_i (r_i + \gamma V_\psi(s'_i) - Q_\theta(s_i, a_i))^2$;
  $\phi \leftarrow \phi - \eta_\pi \nabla_\phi - \frac{1}{B} \sum_i \exp(\beta(Q_{\hat{\theta}}(s_i, a_i) - V_\psi(s_i))) \cdot \log \pi_\phi(a_i|s_i)$;
  Soft-update $\hat{\theta} \leftarrow (1-\rho)\hat{\theta} + \rho\theta$;

Complexity: $O(B \cdot |\theta|)$ per step. Three networks; avoiding explicit Q queries on sampled out-of-data actions removes one source of computation. Actual speed depends on architecture, batch size and implementation.

3.1 Worked numerical example

Take a two-state, two-action MDP. $s \in \{s_0, s_1\}$, $a \in \{0, 1\}$, deterministic transitions $s_0 \xrightarrow{a} s_a$, terminal at s_1 with reward $+1$ on entry, step reward -0.1 on $s_0 \to s_0$. Logger gives $a = 1$ (good action) 70% and $a = 0$ 30%. $\gamma = 0.9$.

True values: $Q^\star(s_0, 0) = -0.1 + 0.9 \cdot Q^\star(s_0, 1) = -0.1 + 0.9 = 0.8$; $Q^\star(s_0, 1) = 1.0$. With $\tau = 0.7$, the expectile $V_\psi(s_0)$ minimises $0.3(1-0.7)(V-0.8)^2 + 0.7(0.7)(1-V)^2$ for $0.8 < V < 1$, giving $V_\psi(s_0) = (0.09 \cdot 0.8 + 0.49 \cdot 1)/(0.09 + 0.49) = 0.96897$, above the logger-weighted average 0.94. The AWR weights are $\exp(\beta(Q - V)) = (\exp(-0.16897\beta), \exp(0.03103\beta))$, so for $\beta = 3$, the relative weight of $a = 1$ vs $a = 0$ is $\exp(0.6) \approx 1.82$ — the relative action probability also includes the logger odds: $(0.7/0.3)\exp(0.6) \approx 4.25$. A fixed temperature is a penalty choice, not a verified hard KL budget. This calculation holds $Q = Q^\star$ fixed; the coupled IQL fixed point need not equal $Q^\star$.

4 Implementation walkthrough

The tabular IQL agent (see `project/agent.py`) has 25 states and 4 actions, so $Q \in \mathbb{R}^{25\times 4}$ and $V \in \mathbb{R}^{25}$.

```
def expectile_loss_grad(u, tau):
    w = np.where(u < 0, 1.0 - tau, tau)
    return w * u

# in IQLAgent.update_value:
u = self.Q[s_idx, a] - self.V[s_idx]
grad = expectile_loss_grad(u, self.tau)
for s, g in zip(s_idx, grad):
    self.V[s] += self.lr_v * g
```

The CQL update applies the standard Bellman step *and* a per-state penalty gradient $\text{softmax}(Q(s,\cdot)) - \mathbf{e}_{a_\beta}$ scaled by α:

```
qs = self.Q[s]
p = np.exp(qs - qs.max()); p /= p.sum()
grad_pen = p.copy(); grad_pen[a_beta] -= 1.0
self.Q[s] -= self.lr_q * self.alpha * grad_pen
```

The dataset comes from `collect_logged_dataset` which mixes an ε-greedy logger ($\varepsilon = 0.3$) with 20% pure-random episodes — imitating a noisy real-world logger.

5 Empirical results

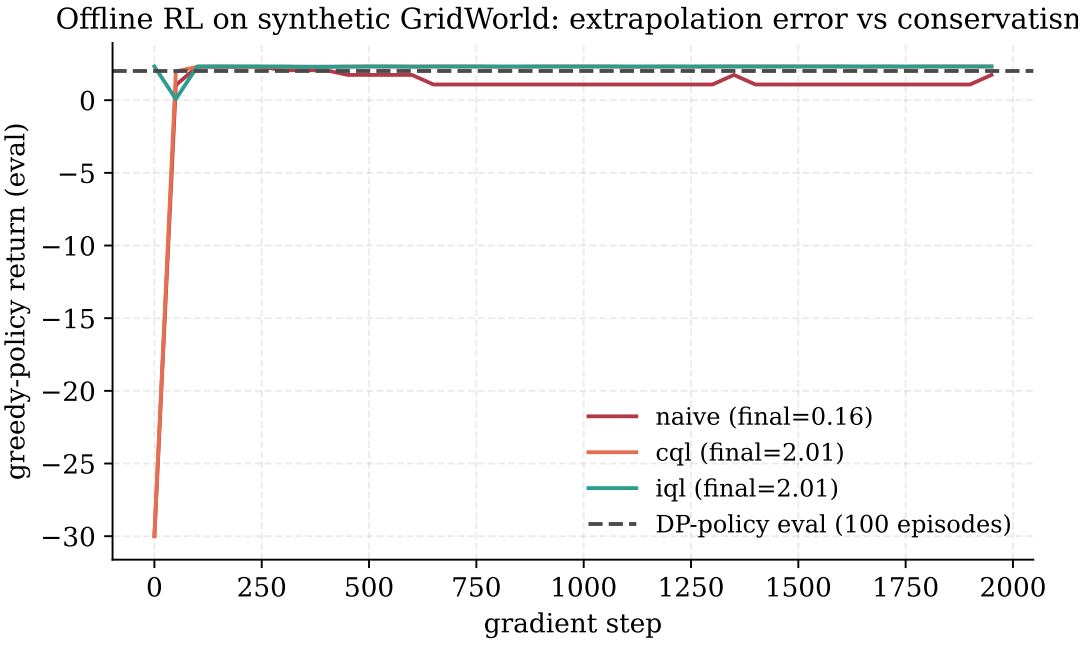


Figure 5.1: Greedy-policy evaluation return on the 5×5 GridWorld. The dashed line evaluates the dynamic-programming policy on 100 episodes. Curves use 200 logged episodes, 2000 updates, and one seed; their ordering is an empirical observation, not a guarantee of any offline algorithm.

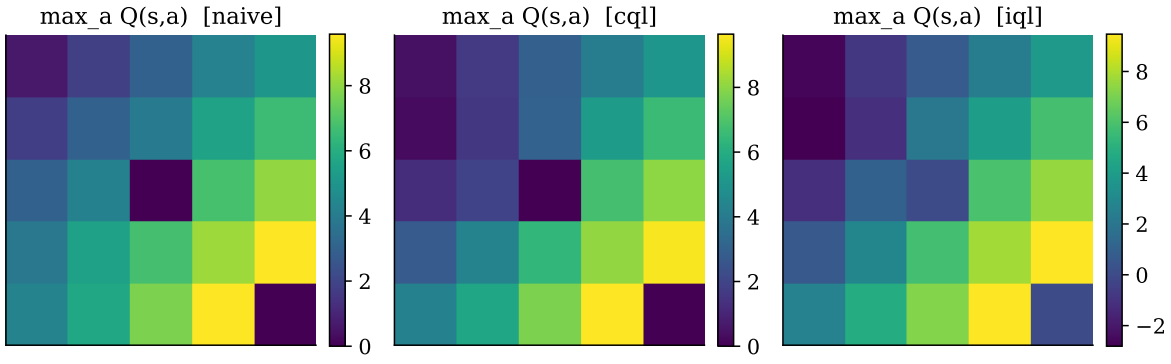


Figure 5.2: Learned $\max_a Q(s, a)$ heatmaps. These show fitted scores, without an independent pointwise error comparison or a certified CQL lower bound.

Safety angle

Distribution-shift failure modes for offline RL in production.

1. **Silent extrapolation.** A naive Q-learner trained on broker–lead logs will happily score lead–broker pairs that never co-occurred. There is no "I don't know" signal; the lift looks plausible until the policy is shipped.

2. **Logger drift.** Changing logging policies need not bias IS if each event records its actual conditional propensity. Using one stale propensity model for all events can invalidate the weights.

3. **Evaluation controls.** Specify target support, effective sample size, uncertainty assumptions and independent evaluation data. A narrow interval does not cure missing overlap or unmeasured confounding. Behaviour penalties and new data collection are separate design choices; neither is an automatic deployment certificate.

Citations: Levine et al. 2020, arXiv:2005.01643 (tutorial); Kumar et al. 2020, NeurIPS (CQL).

Matching application

Synthetic recruitment matching from logs. Consider recorded candidate, job, action and outcome tuples. IQL can illustrate value regression and weighted policy extraction on those logged actions. Its expectile is an asymmetric summary of fitted Q-values, not a confidence bound, a fairness measure or a verified best plausible outcome. A KL penalty controls one notion of policy change; it does not establish explainability or legal compliance. Historical selection, missing outcomes and support need their own analysis.

6 Exercises

See `workbook.ipynb` for the graded, hint-enriched versions.

- **Ex. 1 ($\star$)** Derive and plot the expectile loss for $\tau \in \{0.5, 0.7, 0.9\}$.
- **Ex. 2 ($\star$)** Compute AWR weights for a given advantage vector.
- **Ex. 3 ($\star\star$)** Derive the CQL penalty gradient w.r.t. a single $Q(s, a)$.
- **Ex. 4 ($\star\star$)** Build the logged dataset and inspect coverage.
- **Ex. 5 ($\star\star$)** Train IQL; visualise the learned policy.
- **Ex. 6 ($\star\star$)** CQL ablation over $\alpha \in \{0.1, 1, 10\}$.
- **Ex. 7 ($\star\star\star$)** SNIPS off-policy evaluation against an ε-smoothed greedy target.
- **Ex. 8 ($\star\star\star$, meaningful objective)** Apply IQL to a tiny recruiter–candidate logged dataset (simulated) and produce a ranked candidate list per job.

7 Further reading

See `extended_reading.pdf` for annotated discussions of Levine et al. 2020 (offline RL tutorial), Kumar et al. 2020 (CQL), Kostrikov et al. 2021 (IQL), Tarasov et al. 2023 (ReBRAC), and the OPE survey by Voloshin et al. 2021. Sourced from `research/offline_modelbased_sota.md`.